Informática2 - Desafío2

UdeAWorldCup — Avance 1: Análisis y Diseño Inicial

2026-04-19

Contents

Objetivo 2
Comprensión del problema 2
Identificación de entidades y clases propuestas 2
Relaciones entre clases 3
Diagrama de clases 4
Manejo de memoria 6
Dificultades anticipadas 6
Cosas que aún no están decididas 7

Objetivo

Hacer un análisis sobre el problema antes de iniciar a programar código C++.

Es necesario entender qué entidades existen en el mundo del fútbol que hay que modelar, cómo se relacionan entre sí, y cuál es la mejor manera de organizarlas en clases de C++.

En el análisis también se identifican las dudas y dificultades actuales y que se anticipan para el desarrollo.

Comprensión del problema

El sistema **UdeAWorldCup** debe simular el Mundial de Fútbol FIFA 2026. Esto implica:

- Gestionar **48 equipos**, cada uno con **26 jugadores**.
- Realizar un **sorteo de grupos** respetando restricciones de confederación.
- **Simular 104 partidos** en múltiples etapas, usando una ecuación estadística para calcular los goles.
- Actualizar **estadísticas históricas** de equipos y jugadores luego de cada partido.
- Leer datos iniciales desde un archivo `.csv` y guardar resultados.
- Medir el **consumo de memoria e iteraciones** al finalizar cada funcionalidad.

No se puede usar la STL (nada de `vector`, `list`, `map`, etc.), lo que significa que todas las estructuras de datos hay que construirlas manualmente usando memoria dinámica con `new` y `delete`.

Identificación de entidades y clases propuestas

Después de analizar el problema, identifiqué las siguientes entidades principales que deben representarse como clases:

`Jugador`

Representa a cada uno de los 26 jugadores de un equipo.

Atributos que necesita: - `nombre` y `apellido` (cadenas de texto — `char*`) - `numeroCamiseta` (entero) - Estadísticas históricas: goles, partidos jugados, minutos jugados, asistencias, tarjetas amarillas, tarjetas rojas, faltas

Decisión de diseño: Los nombres se almacenarán como `char*`

`EstadisticasPartidoJugador`

Cuando un jugador participa en un partido específico, se registran sus estadísticas de ese partido en particular (no las históricas). Tiene sentido separar esto en su propia estructura para no mezclar datos históricos con datos de un partido puntual.

Atributos: goles, tarjetas amarillas, tarjetas rojas, faltas, minutos jugados.

`Equipo`

Representa a una selección nacional participante.

Atributos que necesita: - País, confederación, director técnico - Ranking FIFA - Estadísticas históricas del equipo: goles a favor, goles en contra, partidos ganados, empatados, perdidos, tarjetas amarillas, rojas, faltas - Plantilla de 26 jugadores → **arreglo dinámico de punteros a `Jugador`**

Decisión de diseño: La plantilla de jugadores se manejará como un arreglo dinámico `Jugador**` de tamaño fijo (26) que permite acceder a los jugadores directamente por índice.

`EstadisticasPartidoEquipo`

Registra los datos de un equipo en un partido particular: goles a favor, goles en contra, porcentaje de posesión, y cuáles 11 jugadores participaron con sus estadísticas individuales.

`Partido`

Representa un partido del torneo.

Atributos que necesita: - Fecha, hora, sede - Códigos de los 3 árbitros (`char*`) - Referencias a los dos equipos (`Equipo*`) - Estadísticas del partido por equipo (`EstadisticasPartidoEquipo`) - Si se resolvió en prórroga (booleano)

`Grupo`

Agrupa a 4 equipos durante la fase de grupos.

Atributos: identificador de letra (A–L), arreglo de 4 punteros a `Equipo`, puntos acumulados por cada equipo, diferencia de goles.

`Fixture`

Información sobre las fechas de los encuentros. Organiza y controla el flujo completo del torneo. Conoce en qué etapa se está, guarda los partidos de cada ronda y maneja las transiciones entre etapas.

Responsabilidades: - Armar el calendario de la fase de grupos respetando las restricciones de fechas. - Gestionar R16, R8, QF, SF, 3er puesto y Final. - Determinar clasificados de cada etapa.

`Torneo`

Clase principal que coordina todo el sistema. Actúa como punto de entrada y contiene a todos los equipos, grupos.

Tareas: - Cargar datos del CSV. - Inicializar jugadores aleatoriamente. - Ejecutar el sorteo de grupos. - Invocar la simulación de cada etapa. - Generar el informe de estadísticas finales. - Medir y reportar consumo de recursos.

`MedidorRecursos`

Clase auxiliar que lleva la cuenta de iteraciones y calcula el consumo de memoria en cualquier momento de la ejecución.

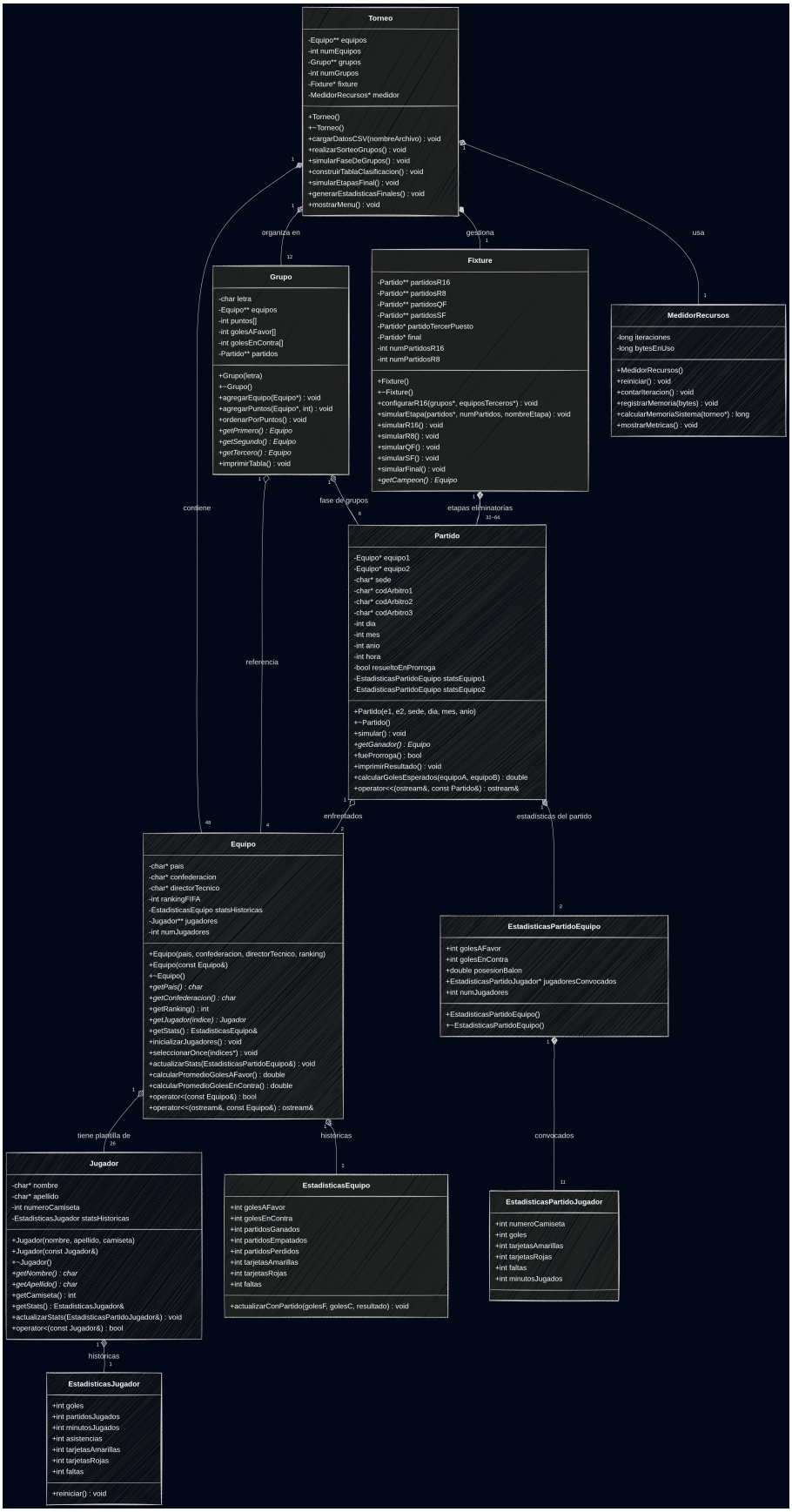
Relaciones entre clases

Relación	Tipo	Descripción
`Torneo` → `Equipo`	Composición	El torneo contiene y gestiona todos los equipos
`Equipo` → `Jugador`	Composición	Cada equipo "posee" su plantilla de jugadores
`Torneo` → `Grupo`	Composición	El torneo contiene los 12 grupos
`Grupo` → `Equipo`	Asociación	El grupo referencia equipos, no los posee
`Torneo` → `Fixture`	Composición	El torneo tiene un fixture
`Fixture` → `Partido`	Composición	El fixture gestiona todos los partidos
`Partido` → `Equipo`	Asociación	El partido referencia dos equipos existentes
`Partido` → `EstadisticasPartidoEquipo`	Composición	El partido contiene las estadísticas de ambos equipos
`EstadisticasPartidoEquipo` → `EstadisticasPartidoJugador`	Composición	Contiene las estadísticas de los 11 jugadores convocados

---

Diagrama de clases

Se implementó la herramienta https://mermaid.js.org/ para crear al diagrama de clases. Se adjunta el archivo '.mmd' para reproducirlo.


Torneo
-Equipo** equipos
-int numEquipos
-Grupo** grupos
-int numGrupos
-Fixture* fixture
-MedidorRecursos* medidor
+Torneo()
+~Torneo()
+cargarDatosCSV(nombreArchivo) : void
+realizarSorteoGrupos() : void
+simularFaseDeGrupos() : void
+construirTablaClasificacion() : void
+simularEtapasFinal() : void
+generarEstadisticasFinales() : void
+mostrarMenu() : void
organiza en
gestiona
usa
contiene
fase de grupos
etapas eliminatorias
referencia
enfrentados
estadísticas del partido
tiene plantilla de
históricas
convocados
Grupo
-char letra
-Equipo** equipos
-int puntos[]
-int golesAFavor[]
-int golesEnContra[]
-Partido** partidos
+Grupo(letra)
+~Grupo()
+agregarEquipo(Equipo*) : void
+agregarPuntos(Equipo*, int) : void
+ordenarPorPuntos() : void
+getPrimero() : Equipo
+getSegundo() : Equipo
+getTercero() : Equipo
+imprimirTabla() : void
Fixture
-Partido** partidosR16
-Partido** partidosR8
-Partido** partidosQF
-Partido** partidosSF
-Partido* partidoTercerPuesto
-Partido* final
-int numPartidosR16
-int numPartidosR8
+Fixture()
+~Fixture()
+configurarR16(grupos*, equiposTerceros*) : void
+simularEtapa(partidos*, numPartidos, nombreEtapa) : void
+simularR16() : void
+simularR8() : void
+simularQF() : void
+simularSF() : void
+simularFinal() : void
+getCampeon() : Equipo
MedidorRecursos
-long iteraciones
-long bytesEnUso
+MedidorRecursos()
+reiniciar() : void
+contarIteracion() : void
+registrarMemoria(bytes) : void
+calcularMemoriaSistema(torneo*) : long
+mostrarMetricas() : void
Partido
-Equipo* equipo1
-Equipo* equipo2
-char* sede
-char* codArbitro1
-char* codArbitro2
-char* codArbitro3
-int dia
-int mes
-int anio
-int hora
-bool resueltoEnProrroga
-EstadisticasPartidoEquipo statsEquipo1
-EstadisticasPartidoEquipo statsEquipo2
+Partido(e1, e2, sede, dia, mes, anio)
+~Partido()
+simular() : void
+getGanador() : Equipo
+fueProrroga() : bool
+imprimirResultado() : void
+calcularGolesEsperados(equipoA, equipoB) : double
+operator<<(ostream&, const Partido&) : ostream&
Equipo
-char* pais
-char* confederacion
-char* directorTecnico
-int rankingFIFA
-EstadisticasEquipo statsHistoricas
-Jugador** jugadores
-int numJugadores
+Equipo(pais, confederacion, directorTecnico, ranking)
+Equipo(const Equipo&)
+~Equipo()
+getPais() : char
+getConfederacion() : char
+getRanking() : int
+getJugador(indice) : Jugador
+getStats() : EstadisticasEquipo&
+inicializarJugadores() : void
+seleccionarOnce(indices*) : void
+actualizarStats(EstadisticasPartidoEquipo&) : void
+calcularPromedioGolesAFavor() : double
+calcularPromedioGolesEnContra() : double
+operator<(const Equipo&) : bool
+operator<<(ostream&, const Equipo&) : ostream&
EstadisticasPartidoEquipo
+int golesAFavor
+int golesEnContra
+double posesionBalon
+EstadisticasPartidoJugador* jugadoresConvocados
+int numJugadores
+EstadisticasPartidoEquipo()
+~EstadisticasPartidoEquipo()
Jugador
-char* nombre
-char* apellido
-int numeroCamiseta
-EstadisticasJugador statsHistoricas
+Jugador(nombre, apellido, camiseta)
+Jugador(const Jugador&)
+~Jugador()
+getNombre() : char
+getApellido() : char
+getCamiseta() : int
+getStats() : EstadisticasJugador&
+actualizarStats(EstadisticasPartidoJugador&) : void
+operator<(const Jugador&) : bool
EstadisticasEquipo
+int golesAFavor
+int golesEnContra
+int partidosGanados
+int partidosEmpatados
+int partidosPerdidos
+int tarjetasAmarillas
+int tarjetasRojas
+int faltas
+actualizarConPartido(golesF, golesC, resultado) : void
EstadisticasPartidoJugador
+int numeroCamiseta
+int goles
+int tarjetasAmarillas
+int tarjetasRojas
+int faltas
+int minutosJugados
EstadisticasJugador
+int goles
+int partidosJugados
+int minutosJugados
+int asistencias
+int tarjetasAmarillas
+int tarjetasRojas
+int faltas
+reiniciar() : void

Manejo de memoria

Se plantea una propuesta para la gestión de memoria.

Memoria para cadenas de texto (`char*`)

Cada nombre, apellido, confederación, etc., se reservará con `new char[longitud + 1]` y se liberará con `delete[]` en el destructor de la clase correspondiente.

Arreglos dinámicos de objetos

- La plantilla de 26 jugadores en `Equipo`: `Jugador** jugadores = new Jugador*[26]`
- Los 48 equipos del torneo: `Equipo** equipos = new Equipo*[48]`
- Los 12 grupos: `Grupo** grupos = new Grupo*[12]`

Destructor obligatorio

Cada clase que use `new` tendrá un destructor que llame `delete[]` sobre todos los arreglos para evitar posibles fugas de memoria. **Esto es algo que aún no tengo muy claro, si esta es la mejor forma de implementar el destructor.**

Dificultades anticipadas

La restricción de confederaciones en el sorteo

El enunciado dice que no pueden quedar dos equipos de la misma confederación en el mismo grupo, **excepto UEFA que puede tener hasta 2**. Implementar este sorteo de forma aleatoria pero respetando estas restricciones puede ser complicado. **Aún no tengo claro cómo implementar esto.**

El calendario de la fase de grupos

Cada día solo puede haber máximo 4 partidos, y ningún equipo puede jugar dos veces en menos de 3 días. Con 72 partidos de grupo (12 grupos $\times$ 6 partidos), organizar fechas respetando ambas restricciones requiere un poco más de análisis de mi parte. **Todavía no tengo clara la lógica exacta.**

La configuración del R16 (dieciseisavos)

La transición de grupos a R16 es específica: los primeros de grupo vs. los terceros clasificados, luego los primeros restantes vs. los cuatro peores segundos, etc. Además, no pueden enfrentarse equipos del mismo grupo. Esta lógica de emparejamiento no está muy clara en este momento.

Lectura del CSV sin librerías externas

Hay que leer el CSV manualmente, carácter por carácter.

> ¿Implementar una función `leerCSV()` que construya los objetos `Equipo` línea por línea?

Medición de memoria

Calcular el consumo real de memoria requiere saber el tamaño exacto de cada objeto en bytes. Se usará `sizeof()`. Aún no tengo claro si esto reflejará el consumo total del programa.

Cosas que aún no están decididas

- **Formato del archivo de jugadores:** El enunciado pide que nosotros definamos el formato. Se está evaluando si usar un archivo de texto plano separado por comas o un formato personalizado más legible.
- **Cómo representar las fechas:** Es necesario tener presente la fecha, pero aún no sé qué formato debo usar para representarla.
- **Sobrecarga de operadores:** El enunciado exige sobrecargar al menos 2 operadores. Pero no tengo claro cómo implementarlos todavía.